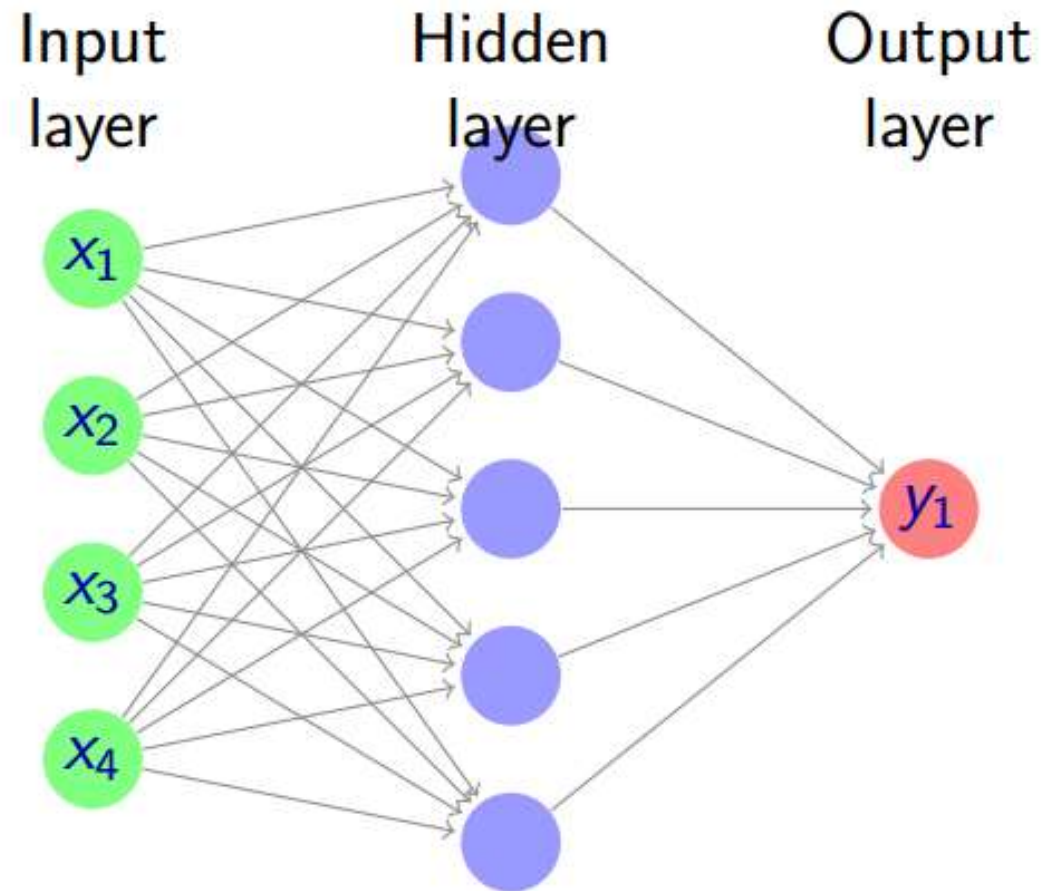


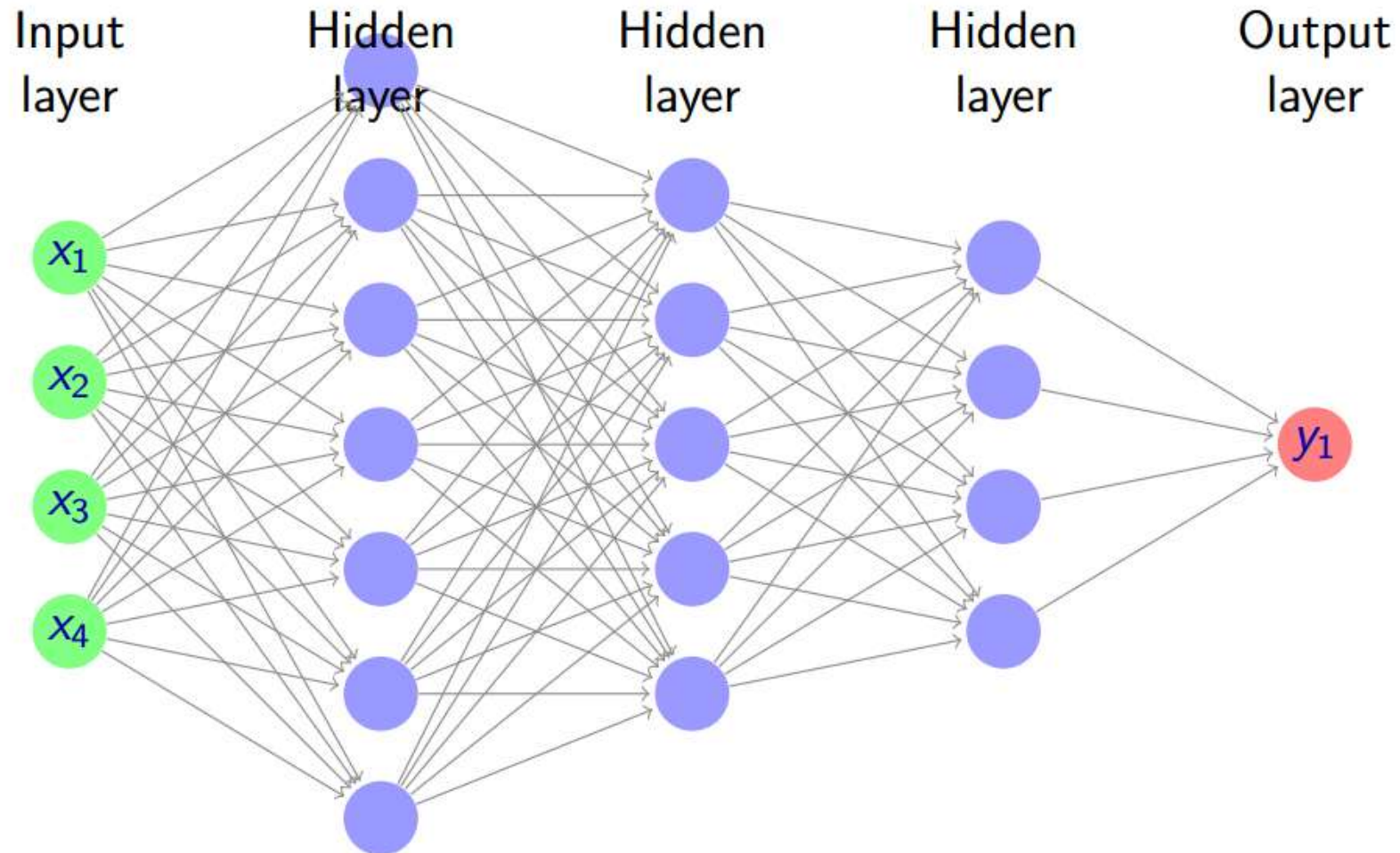
Lecture 7

Feedforward neural networks

Feedforward Neural Network



Feedforward Deep Networks

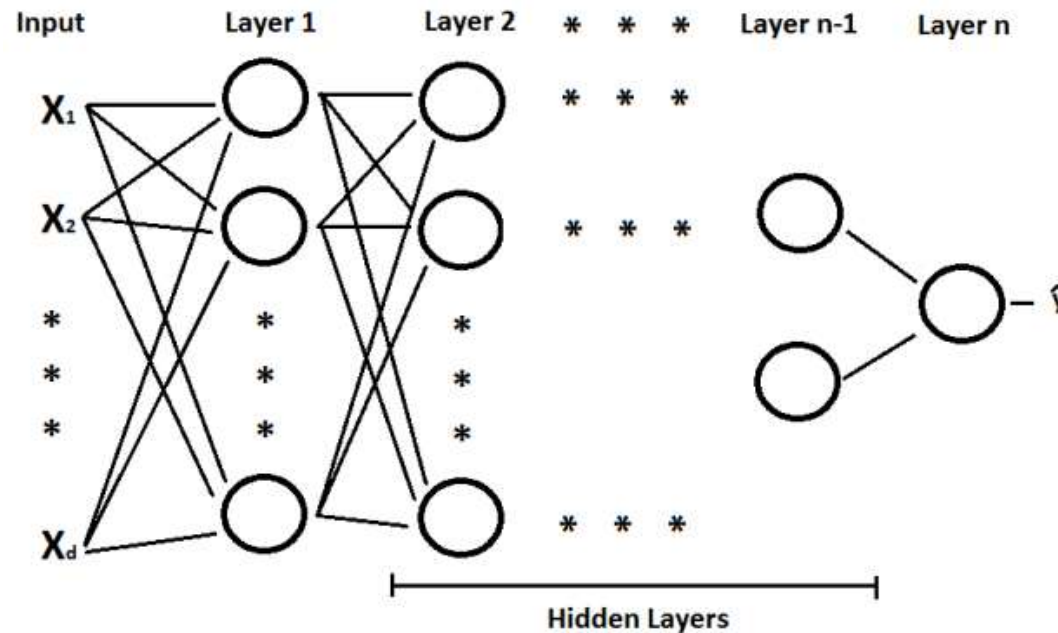


Feedforward Deep Networks

- Feedforward deep networks, a.k.a. multilayer perceptrons (MLPs), are parametric functions composed of several parametric functions.
- Each layer of the network defines one of these sub-functions.
- Each layer (sub-function) has multiple inputs and multiple outputs.
- Each layer composed of many units (scalar output of the layer).
- We sometimes refer to each unit as a feature.
- Each unit is usually a simple transformation of its input.
- The entire network can be very complex.

Feedforward Neural Network

- A neural network is a multistate regression model which is typically represented by a network diagram.



Feed Forward Neural Network

Feedforward Neural Network

- For regression, typically $k = 1$ (the number of nodes in the last layer), there is only one output unit y_1 at the end.
- For c -class classification, there are typically c units at the end with the c^{th} unit modelling the probability of class c , each y_c is coded as 0-1 variable for the c th class.

Feedforward Neural Network

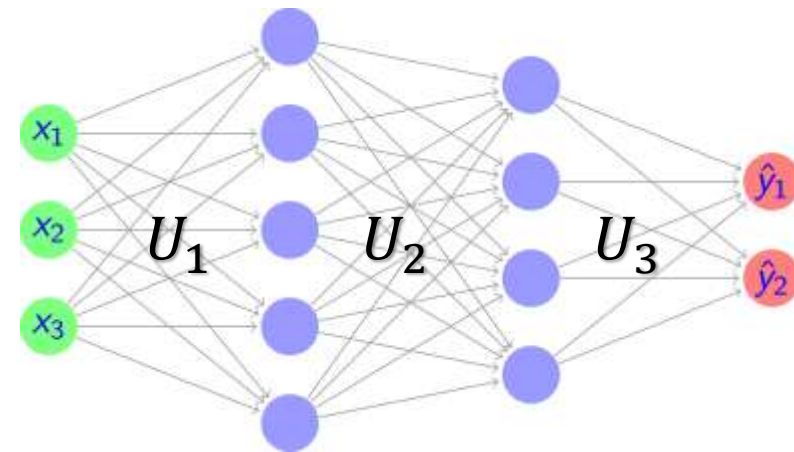
- Consider a feed forward neural network

$$U_1 = \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} & u_{15} \\ u_{21} & u_{22} & u_{23} & u_{24} & u_{25} \\ u_{31} & u_{32} & u_{33} & u_{34} & u_{35} \end{bmatrix}_{3 \times 5}$$

$$U_2 = \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ u_{21} & u_{22} & u_{23} & u_{24} \\ u_{31} & u_{32} & u_{33} & u_{34} \\ u_{41} & u_{42} & u_{43} & u_{44} \\ u_{51} & u_{52} & u_{53} & u_{54} \end{bmatrix}_{5 \times 4}$$

$$U_3 = \begin{bmatrix} u_{11} & u_{12} \\ u_{21} & u_{22} \\ u_{31} & u_{32} \end{bmatrix}_{4 \times 2}$$

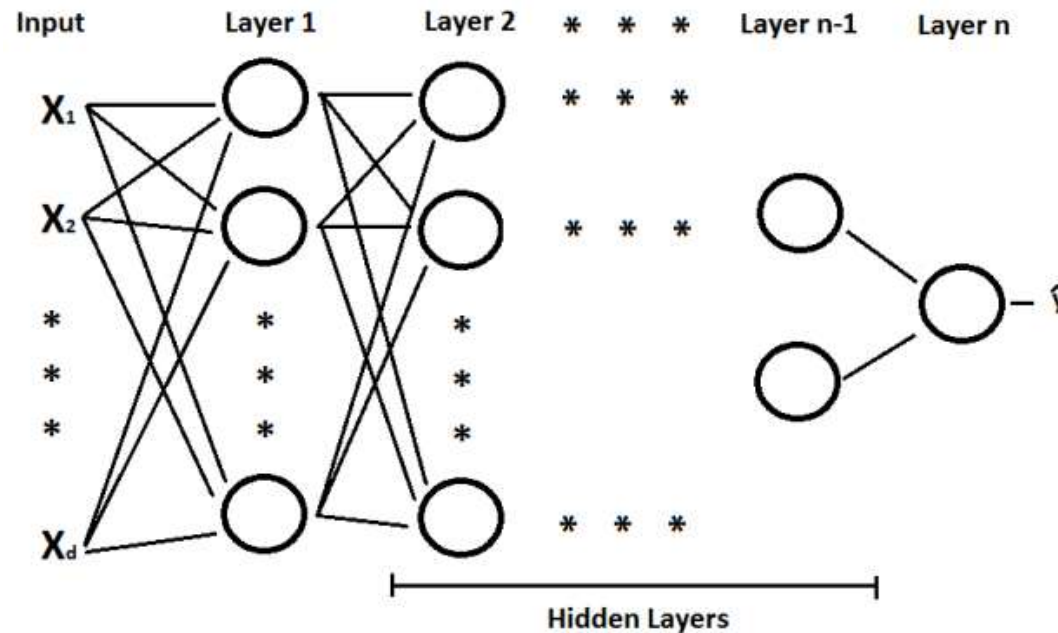
$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$



Backpropagation

Feedforward Neural Network

- A neural network is a multistate regression model which is typically represented by a network diagram.

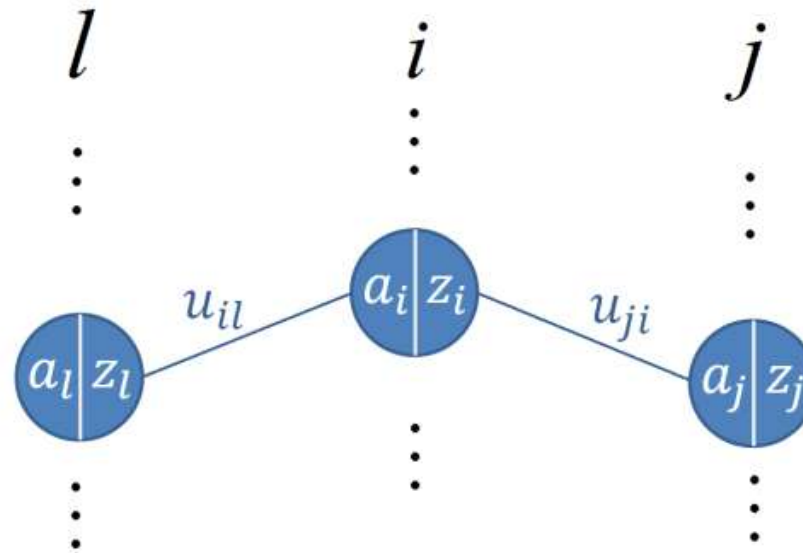


Feed Forward Neural Network

Feedforward Neural Network

- For regression, typically $k = 1$ (the number of nodes in the last layer), there is only one output unit y_1 at the end.
- For c -class classification, there are typically c units at the end with the c^{th} unit modelling the probability of class c , each y_c is coded as 0-1 variable for the c th class.

Backpropagation



Nodes from three hidden layers within the neural network. Each node is divided into the weighted sum of the inputs and the output of the activation function.

$$a_i = \sum_l z_l u_{il}$$

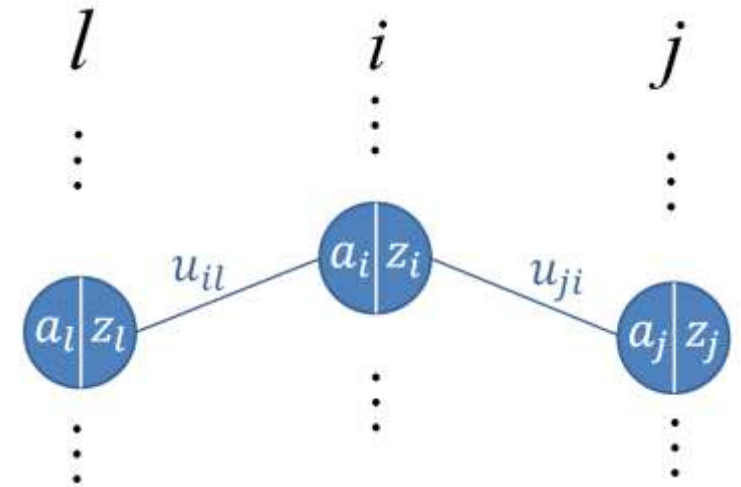
$$z_i = \sigma(a_i)$$

$$\sigma(a) = \frac{1}{1+e^{-a}}$$

- Nodes from three hidden layers within the neural network are considered for the backpropagation algorithm.
- Each node has been divided into the weighted sum of the inputs a and the output of the activation function z . The weights between the nodes are denoted by u .

$$a_i = \sum_l z_l u_{il}$$

$$z_i = \sigma(a_i)$$

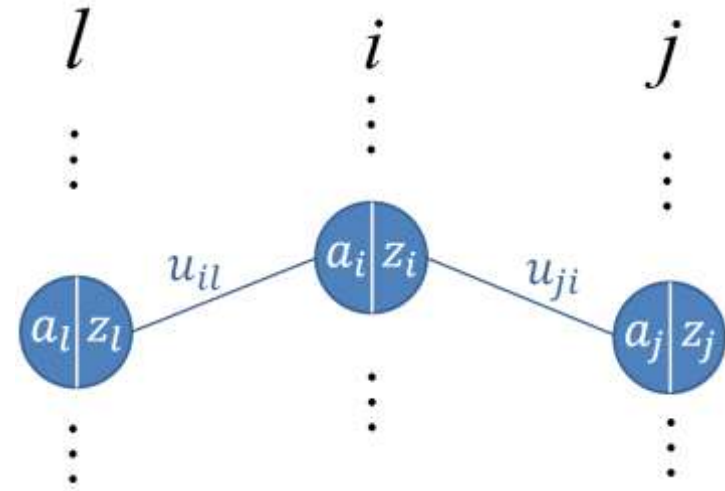


- Take the derivative w.r.t weight u_{il}

$$\frac{\partial |y - \hat{y}|^2}{\partial u_{il}} = \underbrace{\frac{\partial |y - \hat{y}|^2}{\partial a_i}}_{\delta_i} \cdot \underbrace{\frac{\partial a_i}{\partial u_{il}}}_{z_l} \quad \text{Chain rule}$$

$$\frac{\partial |y - \hat{y}|^2}{\partial u_{il}} = \delta_i \cdot z_l$$

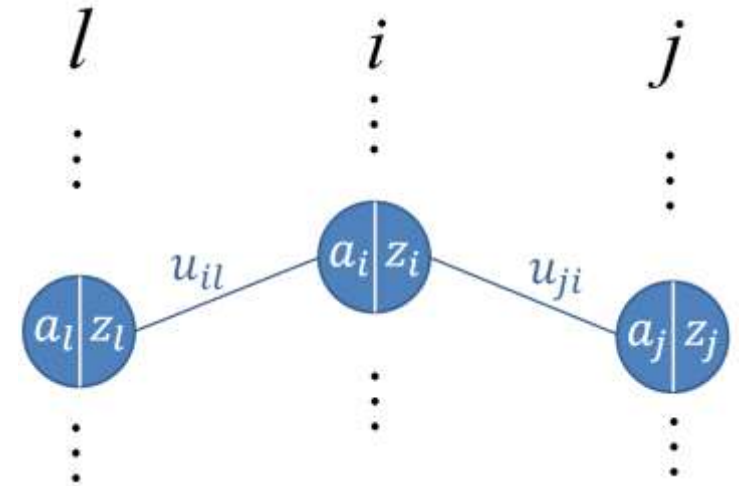
$$\text{where } \delta_i = \frac{\partial |y - \hat{y}|^2}{\partial a_i}$$



- whereas δ_i is unknown but can be expressed as a recursive definition in terms of δ_j .

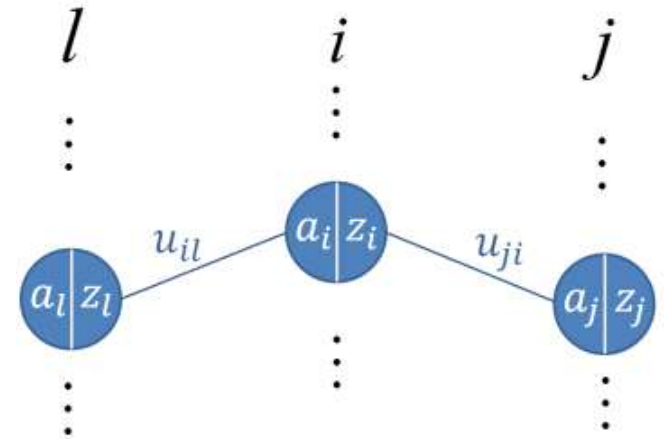
← Chain rule

$$\begin{aligned}
 \delta_i &= \frac{\partial |y - \hat{y}|^2}{\partial a_i} = \sum_j \underbrace{\frac{\partial |y - \hat{y}|^2}{\partial a_j}}_{\delta_j} \cdot \underbrace{\frac{\partial a_j}{\partial a_i}}_{\frac{\partial a_j}{\partial z_i} \cdot \frac{\partial z_i}{\partial a_i}} \\
 \delta_i &= \sum_j \delta_j \cdot \frac{\partial a_j}{\partial z_i} \cdot \frac{\partial z_i}{\partial a_i} \\
 \delta_i &= \sum_j \delta_j \cdot u_{ji} \cdot \sigma'(a_i) \\
 \delta_i &= \sigma'(a_i) \sum_j \delta_j \cdot u_{ji}
 \end{aligned}$$



- The recursive definition of δ_i can be considered as a cost function at layer i for achieving the original goal of optimizing the weights to minimize $\|y - \hat{y}\|^2$

$$\delta_i = \sigma'(a_i) \sum_j \delta_j \cdot u_{ji}$$

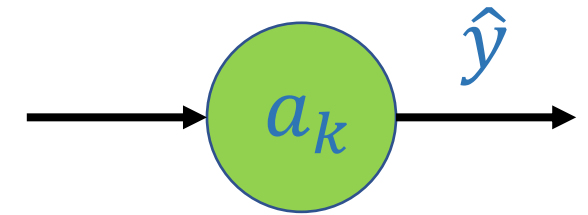


- Now considering δ_k for the output layer:

$$\delta_k = \frac{\partial \|y - \hat{y}\|^2}{\partial a_k}$$

- Where $a_k = \hat{y}$ because an activation function is not applied in the output

$$\delta_k = \frac{\partial \|y - \hat{y}\|^2}{\partial \hat{y}} = -2 \|y - \hat{y}\|$$



- Since y is known and $\hat{y}$ can be computed for each data point.
- assuming small, random, initial values for the weights of the neural network in the beginning.
- Therefore the δ values for the layer before the output layer can be computed using δ_k and then the δ values for the layer before the layer before the output layer can be computed and so on.

$$\delta_i^{(k)} = -2|y_i - \hat{y}_i|$$



$$\delta_i^{(k-1)} = \sigma'(a_i^{(k-1)}) \sum_j \delta_j^{(k)} \cdot u_{ij}^{(k-1)}$$



$$\delta_i^{(k-2)} = \sigma'(a_i^{(k-2)}) \sum_j \delta_j^{(k-1)} \cdot u_{ij}^{(k-2)}$$



- Once all δ values are known, the errors due to each of the weights u will be known and techniques like gradient descent can be used to optimize the weights.

$$u_{il}^{new} \leftarrow u_{il}^{old} - \rho \frac{\partial \|y - \hat{y}\|^2}{\partial u_{il}}$$

Backpropagation

Backpropagation procedure is done using the following steps:

- First arbitrarily choose some random weights (preferably close to zero) for your network.
- Apply x to the FFNN's input layer, and calculate the outputs of all input neurons.
- Propagate the outputs of each hidden layer forward, one hidden layer at a time, and calculate the outputs of all hidden neurons.
- Once x reaches the output layer, calculate the output(s) of all output neuron(s) given the outputs of the previous hidden layer.
- At the output layer, compute $\delta_k = 2(y_k - \hat{y}_k)$ for each output neuron(s).

- Compute each δ_i , starting from $i = k - 1$ all the way to the first hidden layer, where $\delta_i = \sigma'(a_i) \sum_j \delta_j \cdot u_{ji}$.
- Compute $\frac{\partial \|y - \hat{y}\|^2}{\partial u_{il}} = \delta_i z_l$ for all weights u_{il} .
- Then update $u_{il}^{\text{new}} \leftarrow u_{il}^{\text{old}} - \rho \cdot \frac{\partial \|y - \hat{y}\|^2}{\partial u_{il}}$ for all weights u_{il} .
- Continue for next data points and iterate on the training set until weights converge.

Epochs

It is common to cycle through all of the data points multiple times in order to reach convergence. An epoch represents one cycle in which you feed all of your datapoints through the neural network. It is good practice to randomized the order you feed the points to the neural network within each epoch; this can prevent your weights changing in cycles. The number of epochs required for convergence depends greatly on the learning rate & convergence requirements used.

Stochastic gradient descent

Suppose that we want to minimize an objective function that is written as a sum of differentiable functions.

$$Q(w) = \sum_{i=1}^n Q_i(w)$$

Each term Q_i is usually associated with the i -th data point.

Standard gradient descent (batch gradient descent):

$$w = w - \eta \nabla Q(w) = w - \eta \sum_{i=1}^n \nabla Q_i(w)$$

where η is the learning rate (step size).

Stochastic gradient descent

Stochastic gradient descent (SGD) considers only a subset of summand functions at every iteration.

This can be quite effective for large-scale problems.

Bottou, Leon; Bousquet, Olivier (2008). The Tradeoffs of Large Scale Learning. Advances in Neural Information Processing Systems 20. pp. 161168.

The gradient of $Q(w)$ is approximated by a gradient at a single example:
 $w = w - \eta \nabla Q_i(w)$.

This update needs to be done for each training example.

Several passes might be necessary over the training set until the algorithm converges.

η might be adaptive.

Stochastic gradient descent

- Choose an initial value for w and η .
- Repeat until converged
 - Randomly shuffle data points in the training set.
 - For $i = 1, 2, \dots, n$, do:
 - $w = w - \eta \nabla Q_i(w)$.

Example

Suppose $y = w_1 + w_2x$

The objective function is:

$$Q(w) = \sum_{i=1}^n Q_i(w) = \sum_{i=1}^n (w_1 + w_2x_i - y_i)^2.$$

Update rule will become:

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} := \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} - \eta \begin{bmatrix} 2(w_1 + w_2x_i - y_i) \\ 2x_i(w_1 + w_2x_i - y_i) \end{bmatrix}.$$

Example from Wikipedia

Why Stochastic Gradient Descent (SGD) Works

Problem Statement:

- We want to minimize a loss function $Q(\mathbf{w})$ where $\mathbf{w}$ is the parameter vector.
- The loss function is assumed to be differentiable.
- Show that Stochastic Gradient Descent (SGD) converges to a local minimum of $Q(\mathbf{w})$.

Why Stochastic Gradient Descent (SGD) Works

- **Initialization:** Start with an initial guess w_0

- **Update Rule:**

$$w_{\{t+1\}} = w_t - \rho \nabla Q_{i(w_t)}.$$

- **Expectation of the Gradient:**

$$E[\nabla Q_{i(w_t)}] = \nabla Q(w_t).$$

- **Convergence:** If ρ is sufficiently small, then each update moves w_t closer to the minimum of $Q(w)$.

Why Stochastic Gradient Descent (SGD) Works

- Expectation of Update Rule:

$$E[w_{\{t+1\}}] = E[w_t] - \rho E[\nabla Q_{i(w_t)}]$$

- Substitute Expectation of Gradient:

$$E[w_{\{t+1\}}] = E[w_t] - \rho \nabla Q(w_t).$$

- Convergence: This equation implies that the expected value of w moves in the direction of the negative gradient, thus moving towards the minimum of $Q(w)$

Why Stochastic Gradient Descent (SGD) Works

- The proof shows that the expected update of w using SGD is in the direction of the true gradient of the loss function $Q(w)$
- Therefore, SGD works in practice to find a local minimum of the loss function, provided the learning rate ρ is appropriately chosen.

Mini-batches

Batch gradient decent uses all n data points in each iteration.

Stochastic gradient decent uses 1 data point in each iteration.

Mini-batch gradient decent uses b data points in each iteration.

b is a parameter called Mini-batch size.

Mini-batches

- Choose an initial value for w and η .
- Say $b = 10$
- Repeat until converged
 - Randomly shuffle data points in the training set.
 - For $i = 1, 11, 21, \dots, n - 9$, do:
 - $w = w - \eta \sum_{k=i}^{i+9} \nabla Q_i(w)$.

Tuning η

If η is too high, the algorithm diverges.

If η is too low, makes the algorithm slow to converge.

A common practice is to make η_t a decreasing function of the iteration number t . e.g. $\eta_t = \frac{\text{constant1}}{t + \text{constant2}}$

The first iterations cause large changes in the w , while the later ones do only fine-tuning.